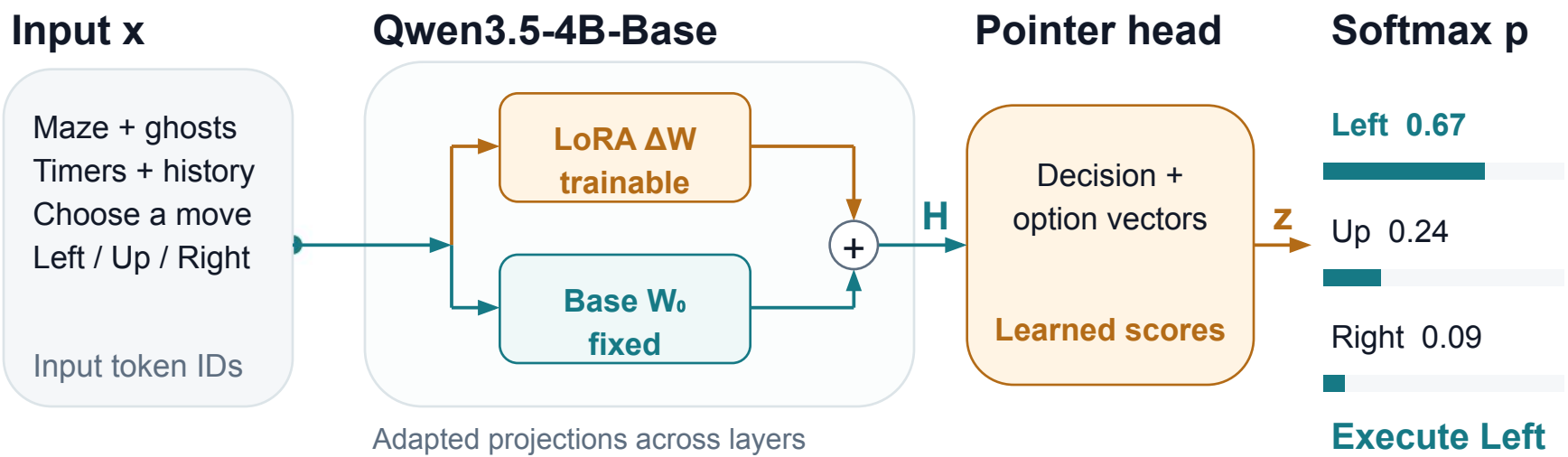
Kev architecture

One board, supplied moves, direct probabilities



$$H = f_\theta(x)$$

$$p = \mathrm{softmax}(z)$$

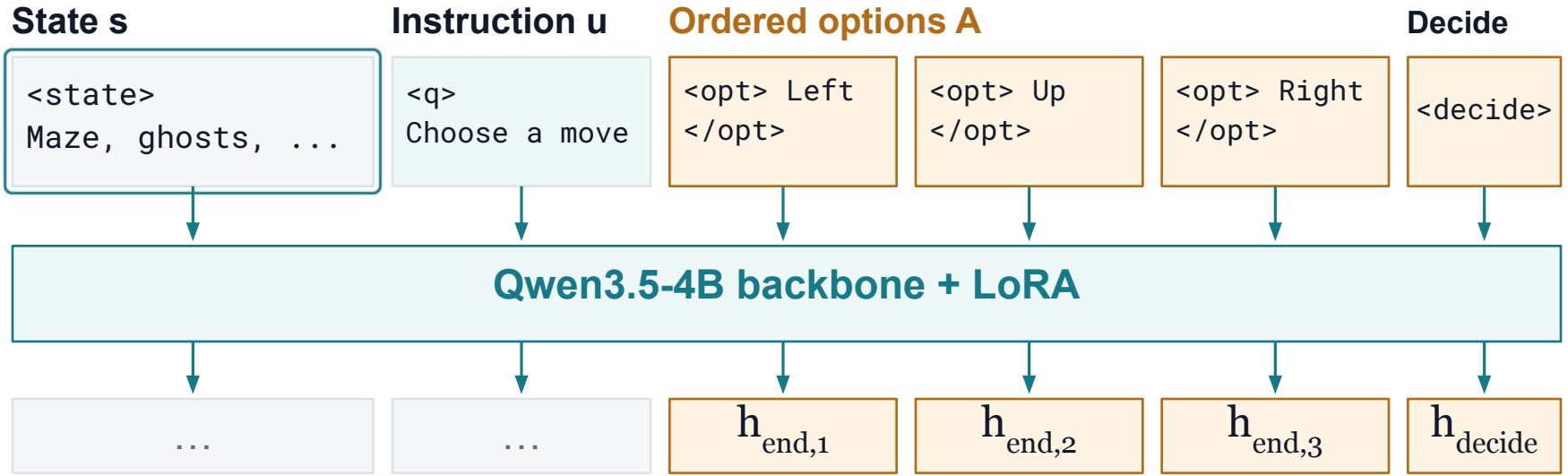
x: input tokens H: hidden vectors θ: effective backbone parameters

W_0: fixed base weights ΔW: LoRA updates z: scores p: probabilities

Illustrative probabilities. Open animation controls.

From board state to hidden vectors

State + instruction + legal moves → one structured input



The pointer head reads each option ending and the final decision vector

$$x = \text{encode}(s, u, A)$$

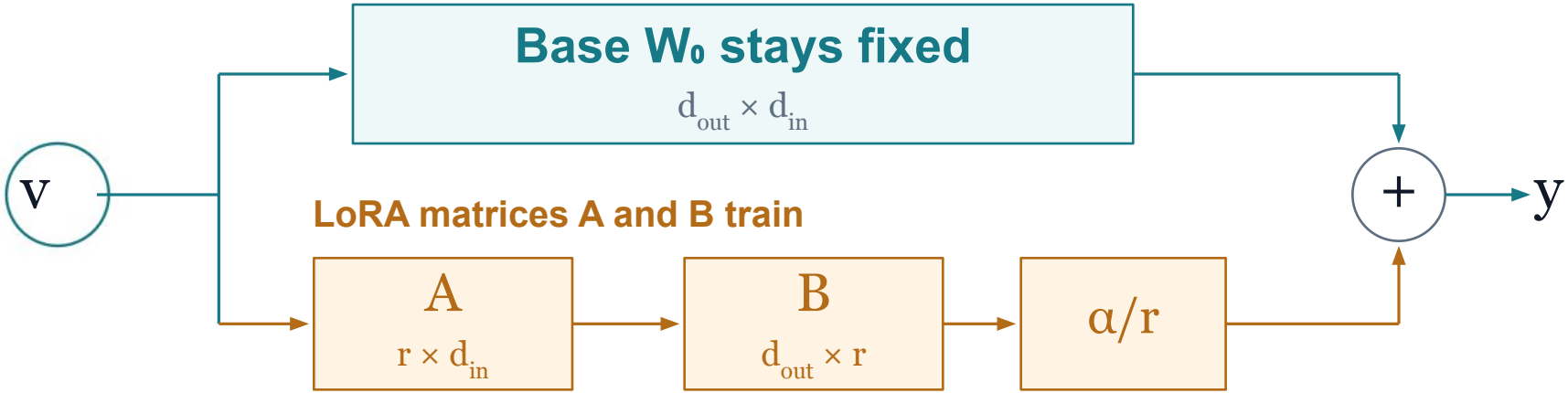
$$H \in \mathbb{R}^{L \times d}$$

L tokens × d features

Schematic token groups. Readable marker names. Open animation controls.

LoRA inside Kev

Small learned matrices adapt each selected projection



$$y = W_o v + (\alpha/r)BAv$$

$$\Delta W = (\alpha/r)BA$$

r = 16 α = 32 α/r = 2 LoRA dropout = 0.05

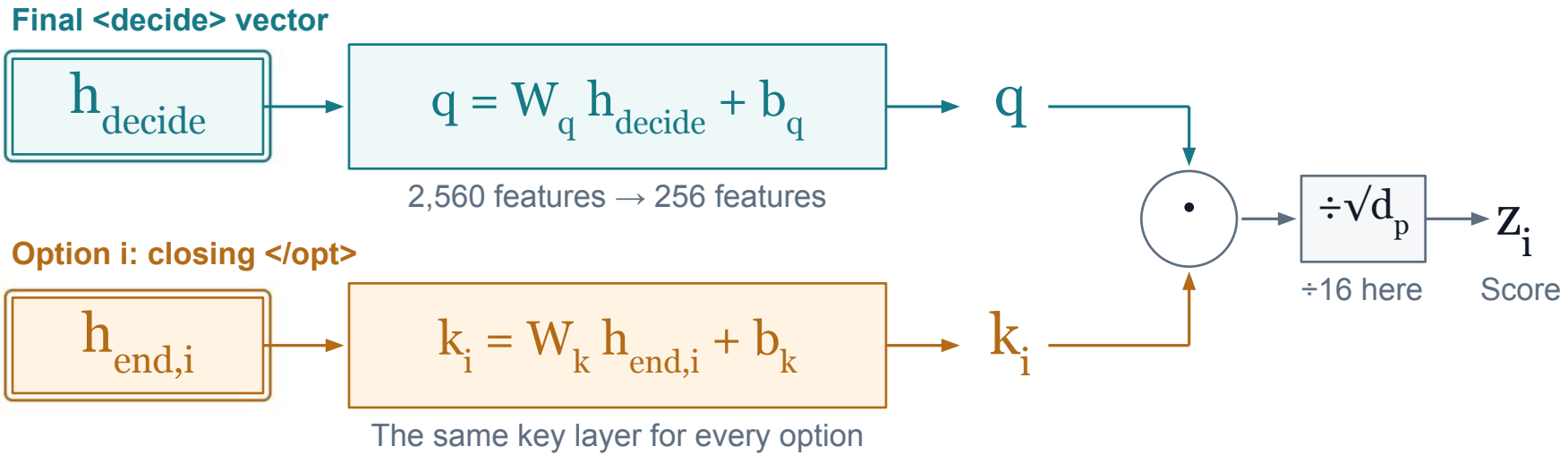
33.8M trainable parameters across LoRA and the pointer head

v: layer input y: layer output d_{in} / d_{out}: input / output widths

One adapted projection. Kev uses all targets. Open animation controls.

The pointer head scores every option

Two learned linear projections, then a scaled dot product



$$z_i = q^T k_i / \sqrt{d_p}$$

K legal options → K raw scores

$d = 2{,}560$ $d_p = 256$ $\sqrt{d_p} = 16$ Both projection layers train

Exact Kev pointer head. i indexes one option. Open animation controls.

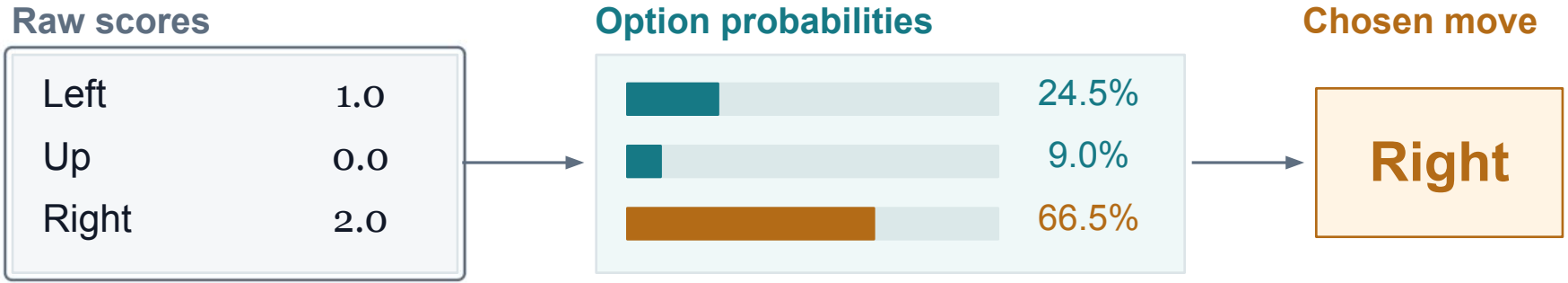
Softmax and action selection

Softmax gives option probabilities. Argmax chooses the largest.

$$p(a_i \mid s) = \frac{\exp(z_i / T)}{\sum_{j=1}^{K} \exp(z_j / T)}$$

$T = 1$

Our fresh lab checkpoint



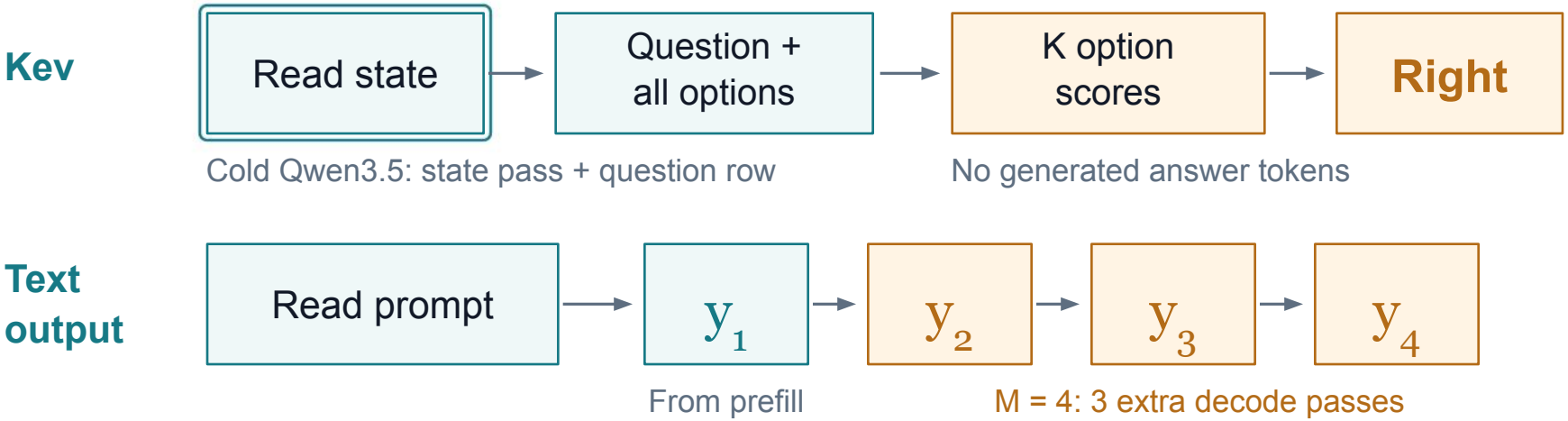
$$i^* = \operatorname{argmax}_i p(a_i \mid s)$$

$$a^* = a_{i^*}$$

Illustrative scores, not a live prediction. Open animation controls.

Why Kev avoids the answer-token loop

The backbone reads the input. The pointer head returns K scores.



$$R_{Kev} \approx (K + 1)\, d\, d_p + K\, d_p$$

$$R_{text} \approx M\, d\, V$$

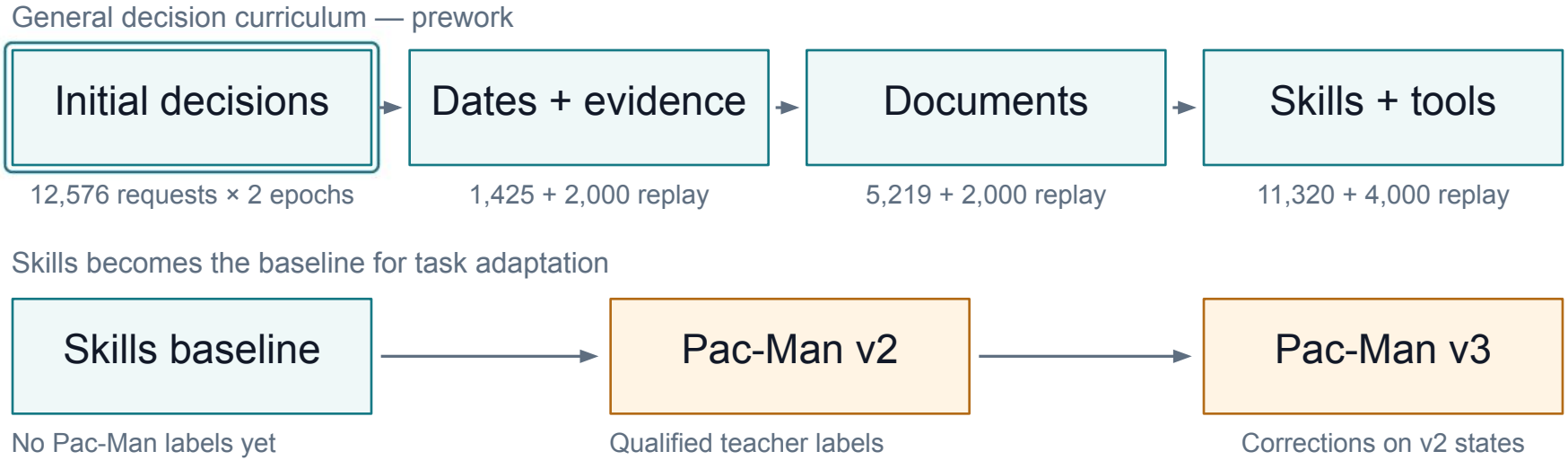
Readout multiplications

K choices, V vocabulary

Illustrative M = 4 answer. Counts exclude backbone work. Open animation controls.

A curriculum, then task checkpoints

Each new stage loads its parent's LoRA and pointer head.



$$\varphi_{\text{child}}^{\,o} = \varphi_{\text{parent}} \qquad D_{\text{train}} = D_{\text{new}} \uplus D_{\text{replay}}$$

New stage: fresh optimizer. Parent checkpoint stays saved.

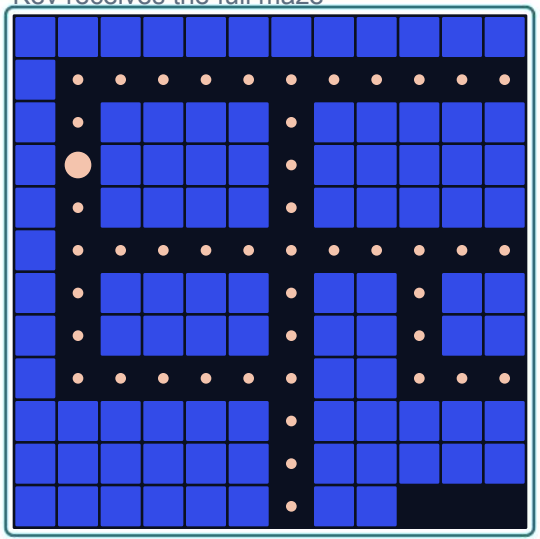
Same fixed Qwen base. Counts name requests; labels are not game wins. Open animation controls.

The game state Kev actually receives

A structured board, moving actors, native timers and memory.

Display crop only

Kev receives the full maze



. pellet · o power pellet

Player + four distinct ghosts

Blinky **Pinky** **Inky** **Clyde**

row/column · heading · pixel_offset · speed_phase

Ghosts: mode · frightened · next/pending turns

Native clocks

timing.power.remaining_frames

chase/scatter · release · Elroy · fruit · eating pauses

Movement and pellet history

recent_positions (up to 64) · destination_visits

decisions_since_last_pellet · visits_to_current_tile

$s_t = O(g_t)$

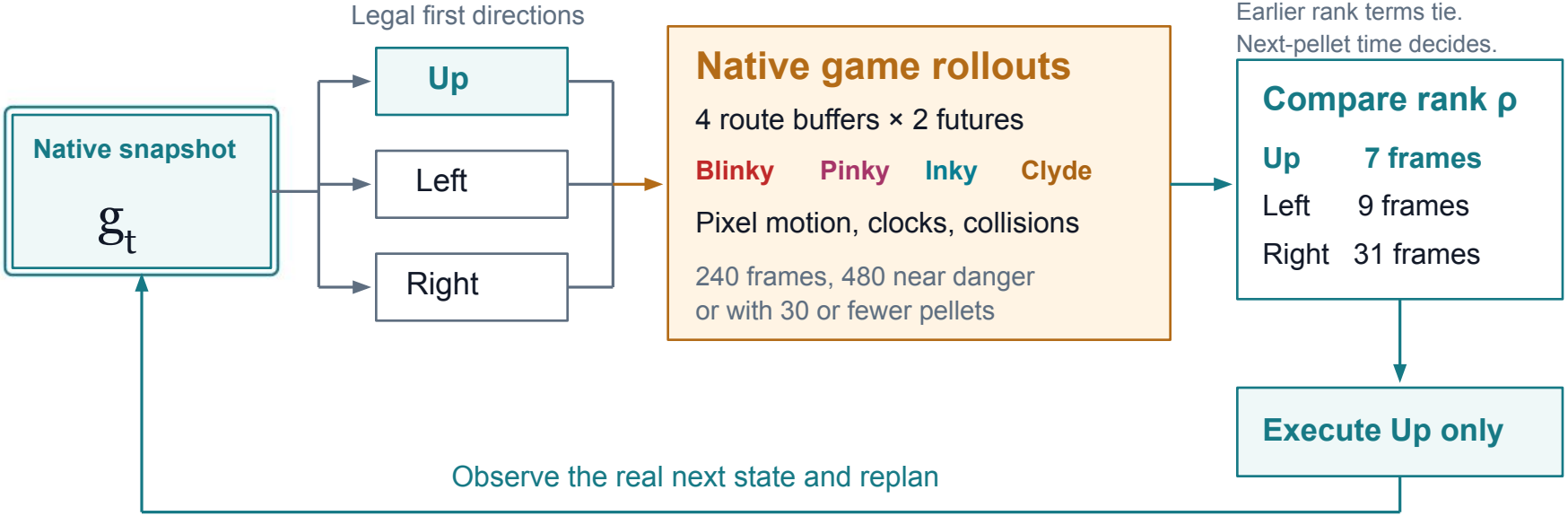
$a_t \in A_t$

Wall-legal directions

Teacher forecasts and labels stay outside this input. Open animation controls.

Native rollout MPC teacher

Compare forecasts, execute one direction, then replan.



$$(a_t^T, b_t^*) = \operatorname{argmin}^{lex} \rho(g_t, a, b)$$

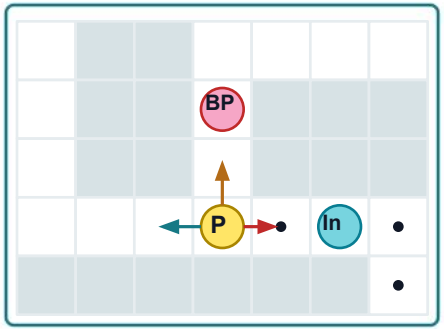
$a \in A_t$
$b \in \{1, 2, 4, 6\}$

Fresh native example. Finite policy portfolio. Open animation controls.

Teacher route planning and future ranking

One native state exposes the trade-off between ghosts, loops and pellets.

Seed 91009, level 1, turn 172



BP: Blinky/Pinky. In: Inky. P: Pac-Man.

Display crop only. Full native game.

First move	Death now	Dry cycle	Next pellet	Alive futures
Left, b=6	**0**	**0**	**137 frames**	**2 / 2, 484 frames**
Up, b=1	0	1	none	0 / 2, 9 frames
Right, b=1	1	0	6 frames	0 / 2, 6 frames

$$\rho = (E, C, O, D, -S, T, K, J, -W, -F, U, \bar{M})$$

First unequal component: Left beats Right at E and Up at C.

Food-route cost is one component of the inner rollout heuristic

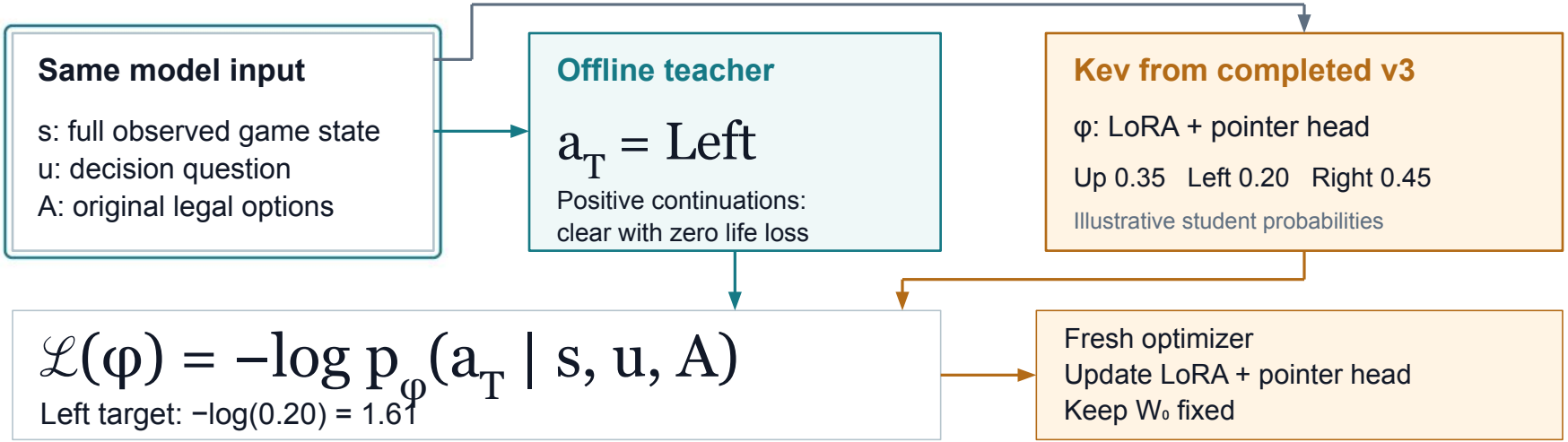
$$J_b(v) = \min_{p \in P} \{d(v,p) + 2\sum_{g \in G} [b - d(g,p)]_+\} + 2N(v)$$

Execute Left only, then replan from the actual next state.

Recorded CPU teacher forecasts. Finite policy portfolio. Open animation controls.

Teacher decisions and Kev distillation

The teacher searches offline. Kev learns its chosen action.



Targeted roots	Nearby decisions	Expert replay	Generic replay	Total requests
3,660	1,220	912	304	6,096

11 roots are recorded v3 disagreements. Other hard cases use simulation.

Probability example only. Seed 91009 stays held out. Open animation controls.

CP1: fine-tune and evaluate Pac-Man

Skills → Pac-Man: rank-16 LoRA + pointer head

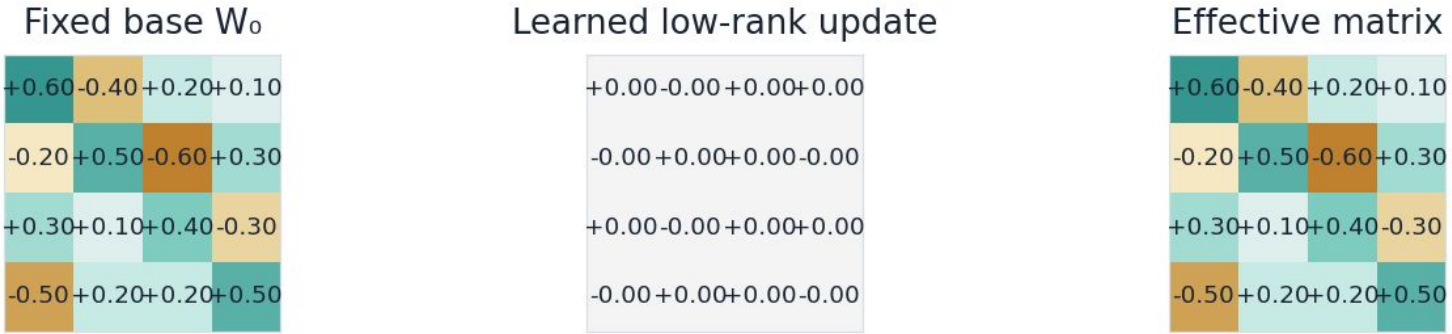
4,096 boards + 2,000 replay; one epoch

762 updates at learning rate 2e-5

$$\mathcal{L}_{\mathrm{CE}} = -\frac{1}{B}\sum_{i=1}^{B} \log p(y_i \mid s_i)$$

B: batch size; s_i: board; y_i: teacher move; p(y_i | s_i): predicted share

LoRA updates and training evidence



Illustrative 4 × 4 matrices, rank 2, α/r = 1. Kev uses rank 16 and α = 32.

$$W_{\text{eff}} = W_0 + \frac{\alpha}{r} BA$$

W0: fixed base; A: r × d_in; B: d_out × r

r = 16; α = 32; 33.8M trainable LoRA/head parameters

Interactive LoRA controls | TensorBoard + recovery; verified Drive backup after each saved snapshot.

Evaluate decisions and gameplay

Same 256 boards: strict and tie-aware agreement

Search regret, survival choices and immediate captures

Same-seed gameplay: pellets, score and revisits

$$A = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}[\hat{a}_i = y_i] \qquad \hat{a}_i = \arg\max_j p(j \mid s_i)$$

N = 256; y_i: teacher move; a_hat: model move; 1[·]: agreement indicator

Submit CP1: show what changed

Explain a changed move or remaining mistake

Connect the move to the ghost behaviour

Show trainable parameters and training curves

Save task adapter and paired evaluation results

Submit the notebook, runtime and compute use

CP1 evidence: reviewed labels + Pac-Man adapter + measured before/after comparison.